Automated CI/CD Pipeline for a Vehicle Service Booking Portal: Jenkins, Docker and Ansible with Automated Quality Gates

Student: Aditya Sinha | **Roll No.:** 23102C0037 | **Topic:** DevOps - CI/CD and Configuration Management

1. Abstract

Small web applications are often released by hand: a developer builds on a laptop, copies an archive to a server and hopes the environment matches. This paper describes a fully scripted delivery workflow for the Vehicle Service Booking Portal (VSBP), a Java 17 / Spring Boot 3.3 application in which customers book and track service appointments and an admin confirms them. A Jenkins declarative pipeline compiles the code, runs the JUnit/Mockito test suite and a Selenium UI gate, packages one WAR and then deploys it three ways: to Tomcat per environment, as a versioned Docker image, and to an Ubuntu node configured by an idempotent Ansible role with one-command rollback. We compare this stack with GitHub Actions, GitLab CI and GitOps/Kubernetes, and propose three low-cost extensions: security scanning, webhook triggers and monitoring with DORA metrics.

2. Introduction

DevOps joins development and operations through automation, fast feedback and shared ownership [1][2]. Research on software-delivery performance links four measures (deployment frequency, lead time, change-failure rate and time to restore) to organisational outcomes [3][13]. Continuous Integration (CI) merges and verifies every change automatically [4]; Continuous Delivery (CD) keeps the verified artefact always deployable [1]. The VSBP repository follows a Git-flow variant (main, develop, feature/*, bugfix/*, ci/*, ops/*) with pull requests, so each merge into develop is a candidate release.

3. Problem Statement

Without automation the VSBP team faces: (a) "works on my machine" drift between dev, staging and production; (b) regressions reaching users because UI journeys such as book, cancel and confirm are checked manually; (c) slow, error-prone deployments with no audit trail of which commit runs where; (d) servers built by hand that cannot be recreated; and (e) no fast way back when a release fails. The goal is a pipeline that turns every commit into a tested, versioned, deployable artefact and a verified running service, and that stops automatically when any check fails.

4. Existing Approaches, Tools and Limitations

Table 1 compares the mainstream options for the CI/CD, packaging and configuration layers.

Approach / tool	Strengths	Limitations for this project
Manual build and copy	No tooling; instant for one developer	Not repeatable, no audit trail, no test gate, human error
Jenkins (pipeline as code) [5]	Free, self-hosted, huge plugin set, Jenkinsfile versioned with code, runs on Windows and Linux agents	Server to maintain; plugins and tool names need care; polling adds latency without a webhook
GitHub Actions [9] / GitLab CI [10]	Hosted runners, YAML next to code, marketplace actions	Minutes quota on private repos; less control of local Tomcat/Docker hosts; lock-in to one platform
Docker [6] vs. VM images	Same image in every environment, layered cache, small JRE runtime	Needs registry and host; persistence needs volumes
Ansible [7] vs. Chef/Puppet	Agentless (SSH), readable YAML, idempotent modules	Slower on very large fleets; push model only
Kubernetes + Argo CD (GitOps) [11][12]	Self-healing, rolling updates, declarative state in Git	Heavy operational cost for a single-service portal
Selenium WebDriver [8]	Real-browser end-to-end checks in CI	Slower and more brittle than unit tests; needs headless Chrome

Key gap: single tools solve one stage each. The project needs one pipeline that couples tests, packaging, containers and server provisioning with fail-fast gating, at a cost a student team can run on one machine.

5. Proposed Technical Solution and Architecture

The solution is a pipeline-as-code design (Figure 1). Git is the single source of truth; Jenkins orchestrates; Maven produces one immutable WAR; Docker and Ansible are two interchangeable delivery targets; Spring Boot Actuator provides the health signal that decides whether a deployment succeeded. Items marked "new" are the proposed extensions beyond the current baseline.

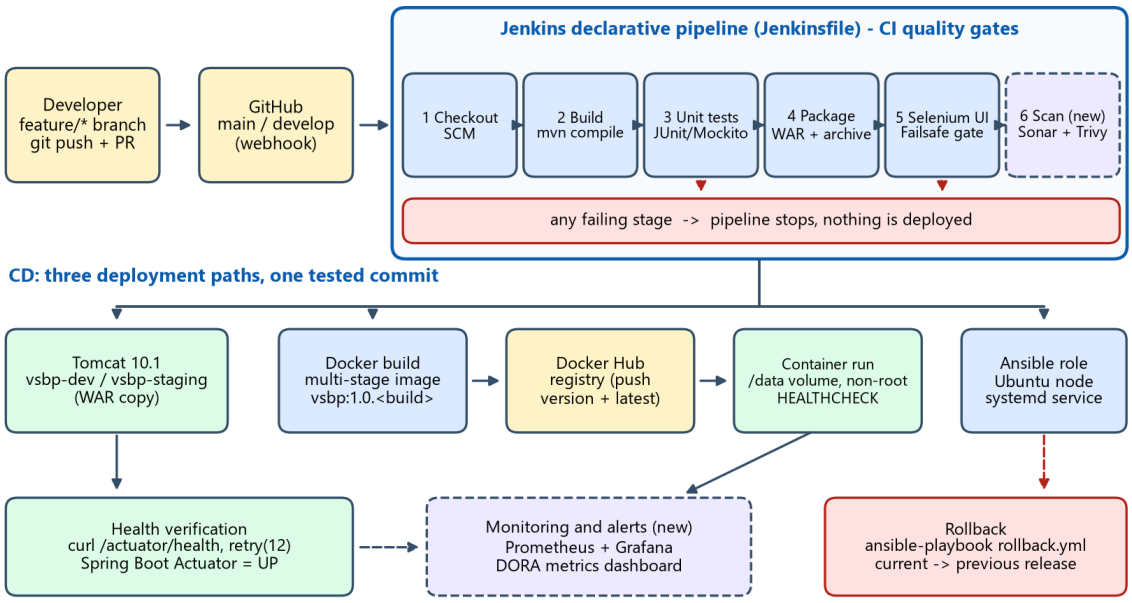


Figure 1. *Architecture of the proposed CI/CD workflow (dashed boxes = proposed additions).*

Design principles. (1) Build once, deploy many: the tested WAR is the artefact, environment name is a pipeline parameter (DEPLOY_ENV = dev | staging | prod). (2) Fail fast: a failing unit or Selenium stage aborts the build before any deploy stage. (3) Everything is code: Jenkinsfile, Dockerfile and Ansible role live in the repository. (4) Verify, do not assume: each deployment polls /actuator/health. (5) Reversible releases: Ansible keeps releases side by side with a current symlink.

6. Implementation Steps

1. **Version control.** Protect main and develop; work on feature/* branches merged by pull request using the repository's issue and PR templates; tag releases.
2. **Jenkins setup.** Install Jenkins LTS on JDK 17, add the Git, Pipeline, JUnit and Docker Pipeline plugins, register tools Maven-3.9 and JDK-17, create a "Pipeline script from SCM" job with an SCM poll (H/2 * * * *) that a GitHub webhook replaces.
3. **CI stages.** Checkout, mvn compile, mvn test (JUnit 5 + Mockito, report published), mvn package -Dapp.environment=<env> with the WAR archived and fingerprinted.
4. **UI quality gate.** Run Selenium WebDriver journeys headless through the Maven Failsafe plugin [8][14]; the goals are bound explicitly so the gate runs on every verify; screenshots are archived on failure.
5. **Deploy to Tomcat.** Copy the WAR to webapps/vsbp-<env>.war and retry curl --fail on /actuator/health up to 12 times [15].
6. **Containerise.** Multi-stage Dockerfile [6]: Maven build stage with a cached dependency layer, then a small JRE-alpine runtime running as a non-root user with a HEALTHCHECK; tag 1.0.<build>, push to Docker Hub, replace the running container with a /data volume.
7. **Provision with Ansible.** Role vsbp installs Java 17, creates the service user and folders, copies the release, templates the environment file and systemd unit, then waits for health. A second run reports no changes (idempotency); rollback.yml re-points current to the previous release [7].

8. **Proposed extensions.** Add SonarQube static analysis and Trivy image scanning as stage 6, switch polling to a webhook, and export Actuator metrics to Prometheus/Grafana with alerts.

7. Output and Results

Running the pipeline on the repository produces: a green stage view per commit; JUnit and Failsafe reports and archived screenshots; a fingerprinted WAR; two parallel Tomcat deployments (/vsbp-dev and /vsbp-staging) each showing its own environment name; a versioned image on Docker Hub; and an Ansible-built node that answers {"status":"UP"} on /actuator/health. A deliberately injected defect (docs/defect-demo) turns the build red at the test stage and nothing is deployed, which demonstrates the gate.

Aspect	Before (manual)	After (pipeline)
Release steps run by a human	Build, copy, restart, check (4+)	1 click or 0 (SCM trigger)
Test gate before deploy	Optional, by memory	Mandatory: unit + Selenium
Rebuild a server	Hours, undocumented	One ansible-playbook run, idempotent
Rollback	Manual re-copy	One playbook: symlink switch + health check

8. Benefits

Shorter lead time and fewer failed releases through early, automatic feedback [3]; identical artefacts across environments, removing configuration drift; a complete audit trail from commit to running container; reproducible infrastructure; and a safe, rehearsed recovery path. The stack is open-source and runs on a single workstation, so cost stays near zero. Limitations: Jenkins needs upkeep, H2 suits demos rather than production, and Selenium suites must be kept small to stay fast.

9. Conclusion

A Jenkins pipeline that combines unit tests, a Selenium gate, a Docker image and an Ansible role gives the VSBP project a repeatable, auditable and reversible delivery process without the overhead of Kubernetes. The proposed security scanning, webhooks and monitoring close the main remaining gaps, and the same Jenkinsfile can later feed a GitOps cluster if the portal grows.

References

[1] J. Humble and D. Farley, Continuous Delivery. Addison-Wesley, 2010.

[2] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook. IT Revolution Press, 2016.

[3] N. Forsgren, J. Humble and G. Kim, Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.

[4] M. Fowler, "Continuous Integration," 2006. https://martinfowler.com/articles/continuousIntegration.html

[5] Jenkins Project, "Pipeline" user documentation. https://www.jenkins.io/doc/book/pipeline/

[6] Docker Inc., "Multi-stage builds" and Dockerfile reference. https://docs.docker.com/build/building/multi-stage/

[7] Red Hat, Ansible Documentation: roles, idempotency and playbooks. https://docs.ansible.com/ansible/latest/

[8] SeleniumHQ, Selenium WebDriver Documentation. https://www.selenium.dev/documentation/

[9] GitHub, GitHub Actions Documentation. https://docs.github.com/en/actions

[10] GitLab, GitLab CI/CD Documentation. https://docs.gitlab.com/ee/ci/

[11] Argo Project, Argo CD Documentation (GitOps). https://argo-cd.readthedocs.io/

[12] The Kubernetes Authors, Kubernetes Documentation. https://kubernetes.io/docs/

[13] DORA (Google Cloud), DevOps Capabilities and Four Key Metrics. https://dora.dev/

[14] Apache Maven, Maven Failsafe Plugin. https://maven.apache.org/surefire/maven-failsafe-plugin/

[15] VMware/Broadcom, Spring Boot Reference: Actuator health endpoints. https://docs.spring.io/spring-boot/reference/actuator/